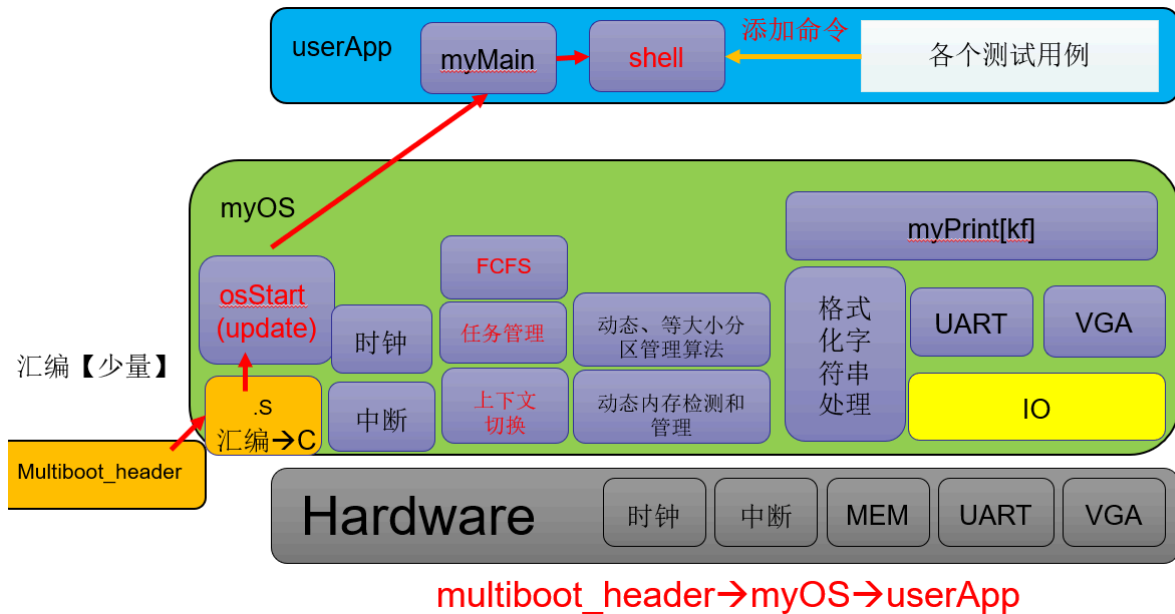


Lab5实验报告

李毅PB22051301

第一部分 系统布局



第二部分 代码解释

1.就绪队列的维护

即为队列基本操作。

```
//TODO:初始化就绪队列（需要填写）
void rqFCFSInit(myTCB* idleTsk) {//对rqFCFS进行初始化处理
    rqFCFS.head = NULL;
    rqFCFS.tail = NULL;
    rqFCFS.idleTsk = idleTsk;
}

//TODO:如果就绪队列为空，返回True（需要填写）
int rqFCFSIsEmpty(void) {//当head和tail均为NULL时，rqFCFS为空
    if(rqFCFS.head == NULL && rqFCFS.tail == NULL)
    {
        return 1;
    }
    else
    {
        return 0;
    }
}

//TODO:获取就绪队列的头结点信息，并返回（需要填写）
myTCB * nextFCFSTsk(void) {//获取下一个Tsk
    if(rqFCFSIsEmpty())
    {
        return rqFCFS.idleTsk;
    }
}
```

```

    }
    else
    {
        myTCB *nextTsk = rqFCFS.head; //获取头结点
        rqFCFS.head = nextTsk->nextTCB; //将头结点出队
        if(rqFCFS.head == NULL) //如果出队后队列为空，将tail也置为NULL
        {
            rqFCFS.tail = NULL;
        }
        return nextTsk;
    }
}

```

//TODO:将一个未在就绪队列中的TCB加入到就绪队列中（需要填写）

```

void tskEnqueueFCFS(myTCB *tsk) { //将tsk入队rqFCFS
    if(tsk->TSK_State != TSK_WAIT)
    {
        tsk->TSK_State = TSK_RDY; //将当前进程状态设置为就绪状态
        tsk->nextTCB = NULL;

        if(rqFCFSIsEmpty())
        {
            rqFCFS.head = tsk;
            rqFCFS.tail = tsk;
        }
        else
        {
            rqFCFS.tail->nextTCB = tsk;
            rqFCFS.tail = tsk;
        }
    }
}

```

//TODO:将就绪队列中的TCB移除（需要填写）

```

void tskDequeueFCFS(myTCB *tsk) { //rqFCFS出队
    if(rqFCFSIsEmpty())
    {
        return;
    }

    if(rqFCFS.head == tsk) //如果要出队的节点是头结点
    {
        rqFCFS.head = tsk->nextTCB;
        if(rqFCFS.head == NULL)
        {
            rqFCFS.tail = NULL;
        }
    }
    else
    {
        myTCB *prev = rqFCFS.head;
        while(prev->nextTCB != NULL && prev->nextTCB != tsk)
        {
            prev = prev->nextTCB; //遍历找到前一个节点
        }
    }
}

```

```

        if(prev->nextTCB == tsk)
        {
            prev->nextTCB = tsk->nextTCB;
            if(rqFCFS.tail == tsk) //如果出队的是尾结点，更新tail
            {
                rqFCFS.tail = prev;
            }
        }
    }
}

```

2.任务池中任务的维护

//TODO:以tskBody为参数在进程池中创建一个进程，并调用tskStart函数，将其加入就绪队列（需要填写）

int createTsk(void (*tskBody)(void)){//在进程池中创建一个进程，并把该进程加入到rqFCFS队列中

```

    if(firstFreeTsk == NULL) //没有可用的TCB
    {
        myPrintk(0x2, "No more TCB available!\n");
        return -1;
    }

    myTCB *newTsk = firstFreeTsk;
    firstFreeTsk = firstFreeTsk->nextTCB;

    newTsk->task_entrance = tskBody; //设置任务入口
    newTsk->TSK_State = TSK_NONE;
    stack_init(&(newTsk->stkTop), tskBody);

    tskStart(newTsk);

    return newTsk->TSK_ID;
}

```

//TODO:以takIndex为关键字，在进程池中寻找并销毁takIndex对应的进程（需要填写）

void destroyTsk(int takIndex) { //在进程中寻找TSK_ID为takIndex的进程，并销毁该进程

```

    if(takIndex < 0 || takIndex >= TASK_NUM) //检查索引是否有效
    {
        myPrintk(0x2, "Invalid task index!\n");
        return;
    }

    myTCB *thisTCB = &tcbPool[takIndex];
    if(thisTCB->TSK_State == TSK_NONE) //如果该进程未分配
    {
        myPrintk(0x2, "Task %d is not allocated!\n", takIndex);
        return;
    }

    thisTCB->TSK_State = TSK_NONE;
    thisTCB->nextTCB = firstFreeTsk;
    firstFreeTsk = thisTCB;
}

```

第三部分 编译过程进行说明

src文件夹结构如下:

```
liyi@MSI:~/OSLab/Lab5/src$ tree
.
├── Makefile
├── Makefile:Zone.Identifier
├── multibootheader
│   ├── multibootHeader.S
│   └── multibootHeader.S:Zone.Identifier
├── myOS
│   ├── Makefile
│   ├── Makefile:Zone.Identifier
│   ├── dev
│   │   ├── Makefile
│   │   ├── Makefile:Zone.Identifier
│   │   ├── i8253.c
│   │   ├── i8253.c:Zone.Identifier
│   │   ├── i8259A.c
│   │   ├── i8259A.c:Zone.Identifier
│   │   ├── uart.c
│   │   ├── uart.c:Zone.Identifier
│   │   ├── vga.c
│   │   └── vga.c:Zone.Identifier
│   ├── i386
│   │   ├── CTX_SW.S
│   │   ├── CTX_SW.S:Zone.Identifier
│   │   ├── Makefile
│   │   ├── Makefile:Zone.Identifier
│   │   ├── io.c
│   │   ├── io.c:Zone.Identifier
│   │   ├── irq.S
│   │   ├── irq.S:Zone.Identifier
│   │   ├── irqs.c
│   │   └── irqs.c:Zone.Identifier
│   ├── include
│   │   ├── i8253.h
│   │   ├── i8253.h:Zone.Identifier
│   │   ├── i8259.h
│   │   ├── i8259.h:Zone.Identifier
│   │   ├── io.h
│   │   ├── io.h:Zone.Identifier
│   │   ├── irq.h
│   │   ├── irq.h:Zone.Identifier
│   │   ├── kmalloc.h
│   │   ├── kmalloc.h:Zone.Identifier
│   │   ├── malloc.h
│   │   ├── malloc.h:Zone.Identifier
│   │   ├── mem.h
│   │   ├── mem.h:Zone.Identifier
│   │   ├── myPrintk.h
│   │   ├── myPrintk.h:Zone.Identifier
│   │   ├── string.h
│   │   └── string.h:Zone.Identifier
```

```

| | └─ task.h
| | └─ task.h:Zone.Identifier
| | └─ uart.h
| | └─ uart.h:Zone.Identifier
| | └─ vga.h
| | └─ vga.h:Zone.Identifier
| | └─ vsprintf.h
| | └─ vsprintf.h:Zone.Identifier
| | └─ wallClock.h
| | └─ wallClock.h:Zone.Identifier
| └─ kernel
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ mem
| | | └─ Makefile
| | | └─ Makefile:Zone.Identifier
| | | └─ dPartition.c
| | | └─ dPartition.c:Zone.Identifier
| | | └─ eFPartition.c
| | | └─ eFPartition.c:Zone.Identifier
| | | └─ malloc.c
| | | └─ malloc.c:Zone.Identifier
| | | └─ pMemInit.c
| | | └─ pMemInit.c:Zone.Identifier
| | └─ task.c
| | └─ task.c:Zone.Identifier
| | └─ tick.c
| | └─ tick.c:Zone.Identifier
| | └─ wallClock.c
| | └─ wallClock.c:Zone.Identifier
| └─ lib
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ string.c
| | └─ string.c:Zone.Identifier
| └─ myOS.ld
| └─ myOS.ld:Zone.Identifier
| └─ osStart.c
| └─ osStart.c:Zone.Identifier
| └─ printk
| | └─ Makefile
| | └─ Makefile:Zone.Identifier
| | └─ myPrintk.c
| | └─ myPrintk.c:Zone.Identifier
| | └─ types.h
| | └─ types.h:Zone.Identifier
| | └─ vsprintf.c
| | └─ vsprintf.c:Zone.Identifier
| └─ start32.S
| └─ start32.S:Zone.Identifier
| └─ userInterface.h
| └─ userInterface.h:Zone.Identifier
└─ output
  └─ multibootheader
    └─ multibootHeader.o
  └─ myOS

```


使用 screen 命令进入交互界面（与 QEMU） `sudo screen /dev/pts/x`（x为生成的对应值）

第四部分 运行结果



成功实现上下文切换以及 FCFS 算法

第五部分 思考题

1. 在上下文切换的现场维护中，pushf 和 popf 对应，pusha 和 popa 对应，call 和 ret 对应，但是为什么 CTS SW 函数中只有 ret 而没有 call 呢？

```
CTX_SW:
    pushf
    pusha

    movl prevTSK_StackPtr, %eax
    movl %esp, (%eax)
    movl nextTSK_StackPtr, %esp

    popa
    popf
    ret
```

这个 ret 不是从 CTX_SW 返回到调度器，而是从新任务栈上弹出之前保存的 EIP，跳回到该任务的执行点。如果在 CTX_SW 里再做一个 call，会往当前（已切换到新任务）栈上再压一个返回地址，破坏这个栈原本保存的任务返回点。

2. 谈一谈你对 stack init 函数的理解。

stack_init 函数的主要作用是为新创建的任务构造一个“初始栈帧”，使得任务可以通过上下文切换机制（例如 CTX_SW）正确恢复并开始执行。其依次完成了以下过程：

- 先将 tskEnd() 的地址压入，当任务执行结束时自动调用，用于完成任务退出；
- 然后将任务函数的入口地址放入 EIP 位置（作为 ret 指令跳转目标）；

- 接着压入 EFLAGS 与通用寄存器，这样当 CTX_SW 对新任务进行恢复（执行 popa、popf、ret）时，一切都能够按照预期，从进入任务运行环境开始到任务结束后自动调用 tskEnd() 来进行清理。

这样设计的好处是在任务创建后，不需要在任务函数内部显式调用 tskEnd()，而是在任务退出时自动进入清理流程。并对首次上下文切换的恢复提供了完整的现场，使得切换过程中不会出现栈下溢或数据不一致的情况。

3. myTCB 结构体定义中的 stack[STACK_SIZE] 的作用是什么？ BspContextBase[STACK_SIZE]的作用又是什么？

myTCB 结构体里的

```
unsigned long stack[STACK_SIZE];
```

为每个任务分配的一块“专属栈空间”。在任务运行期间，任务的局部变量、函数调用帧、以及被抢占时保存的寄存器，都保存在这块 stack 中。

```
unsigned long BspContextBase[STACK_SIZE];
```

则是为“引导态”（BSP，即多任务启动前的主线程）临时准备的一块栈空间。第一次调用 CTX_SW 时就能把引导态的 ESP 保存到 BspContextBase 中。等多任务结束后，如果需要恢复单一内核态继续运行，就能从这块缓冲区中恢复原来的栈。

4. prevTSK_StackPtr 是一级指针还是二级指针？为什么？

prevTSK_StackPtr 用来保存“上一个任务”的栈顶指针地址。故为二级指针。